Project Final Report

Junhyeon Song (Email: songjunh@usc.edu, USC ID: 8467-4177-80)

1. INTRODUCTION & MOTIVATION

Modern ticket reservation platforms (Ticketmaster, SeatGeek, and their peers) must handle thousands of concurrent seat-reservation requests in milliseconds while guaranteeing that no two users successfully book the same seat. These systems exemplify the two core challenges of any high-concurrency transactional database: maintaining correctness under simultaneous writes and sustaining read throughput without sacrificing writers.

ConcertRush is a Python CLI application built to surface the internal mechanics that make PostgreSQL uniquely suited to this class of workload. Rather than treating the database as an opaque query engine, ConcertRush instruments three tightly scoped scenarios — each one designed to trigger, observe, and measure a specific PostgreSQL internal: Multi-Version Concurrency Control (MVCC), heap and index bloat, and VACUUM-driven I/O optimization.

PostgreSQL was chosen because it exposes its internals through a rich set of system views and extensions (pg_stat_user_tables, pgstattuple, heap_page_items, EXPLAIN ANALYZE) that are rarely available at this level of detail in commercial databases. This transparency makes PostgreSQL uniquely educational as a research artifact.

2. DATABASE SYSTEM OVERVIEW

PostgreSQL is an open-source, ACID-compliant relational database with over 35 years of active development. Its architecture is built around a process-based concurrency model and a heap-based storage engine that physically separates table data, index structures, and concurrency metadata.

2.1 High-Level Architecture

PostgreSQL's architecture consists of four tightly coupled subsystems:

- Shared Memory: Shared Buffer Cache (default 128 MB), WAL Buffers, and Lock Tables reside here and are shared across all backend processes.
- Process Model: Each client connection spawns a dedicated backend process. The Postmaster (supervisor) manages lifecycle — forking workers, autovacuum launchers, and WAL writers.
- Storage Layer (Heap): Table data is stored in 8 KB pages (blocks) on disk. Each page holds tuple headers and tuple data. This is called the "Heap" — an unordered pile of versions.
- Indexing Layer: Secondary B-Tree indexes (default) store physical tuple identifiers (ctid = page number + offset) alongside indexed key values.

2.2 Intended Use Cases

PostgreSQL is well-suited to OLTP workloads requiring strict transactional guarantees (banking, ticketing, e-commerce), analytics workloads benefiting from its advanced join strategies and parallel query execution, geospatial applications via PostGIS, and any environment where extensibility or strict SQL standards compliance is required

3. INTERNAL ARCHITECTURE

3.1 MVCC — Multi-Version Concurrency Control

MVCC is PostgreSQL's fundamental mechanism for achieving Snapshot Isolation without requiring readers to wait for writers, and vice versa. It is implemented entirely within the tuple header of every row stored in the heap.

- **Visibility rule (Snapshot Isolation):** When a transaction with snapshot XID=100 reads a tuple, PostgreSQL evaluates whether a tuple is visible if (xmin is committed) AND (xmax is 0 OR xmax is not yet committed OR xmax > 100). This check happens per-tuple, per-read, with no locking required. The result: readers and writers never block each other.

Field	Type	Role
`xmin`	TransactionId	XID of the INSERT transaction. Row is visible only if xmin is committed.
`xmax`	TransactionId	XID of the DELETE/UPDATE transaction. Zero means the row is live.
`ctid`	(page, offset)	Physical location of this version. Changes on every non-HOT update.
`t_infomask`	uint16 bitmask	Caches commit/abort status of xmin/xmax, avoiding repeated clog lookups.

- **Visibility Map:** PostgreSQL maintains a Visibility Map (VM, one bit per heap page) indicating whether all tuples on that page are visible to all current transactions. If the VM bit is set for a page, Index-Only Scans can return data directly from the index without fetching the heap page at all, dramatically reducing I/O.

3.2 Storage Consequences — Heap & Index Bloat

Because updates never overwrite in place, every `UPDATE seats SET status='reserved'` appends a new tuple and leaves a dead tuple behind. In a high-churn reservation workload, this accumulates into measurable bloat.

- **Heap Bloat (Dead Tuples):** Dead tuples persist on heap pages, consuming space. Quantified via `pgstattuple` (dead_tuple_count, dead_tuple_percent).
- **Index Bloat (Non-HOT Updates):** The default secondary index type is B-Tree. Each leaf entry stores the indexed key value alongside the physical ctid of the corresponding heap tuple. A HOT (Heap-Only Tuple) update avoids creating a new index entry only when the updated column is not indexed and the new tuple fits on the same page. Since `status` is indexed in ConcertRush, every seat update is a Non-HOT update, generating a new B-Tree entry per reservation, causing index growth proportional to write volume.

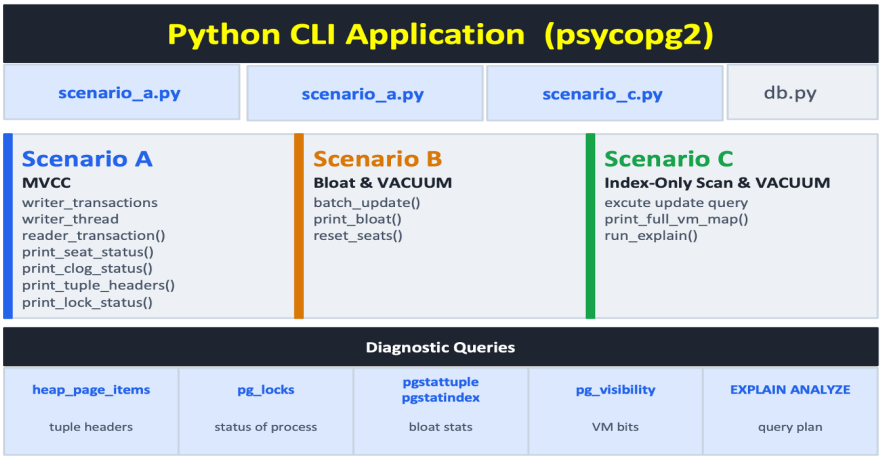
3.3 VACUUM — Garbage Collection & Read Optimization

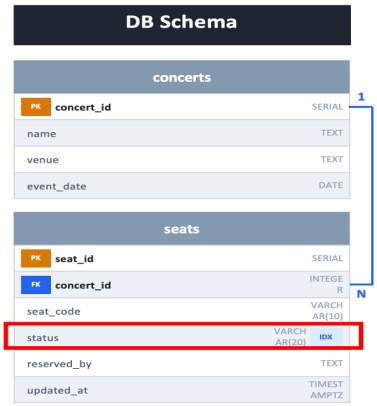
- **Standard VACUUM:** scans heap pages, marks space occupied by dead tuples (xmax set and committed) as reusable, and updates the Visibility Map (VM). The VM is a one-bit-per-page bitmap: when a page's VM bit is set, all tuples on that page are known to be visible to all current and future transactions.
- **Performance payoff:** After VACUUM sets the VM bit, a query using an index on `status` can execute an Index-Only Scan, returning results without fetching the heap page at all. `EXPLAIN ANALYZE` will show Heap Fetches drop from N→0.

4. APPLICATION DESIGN

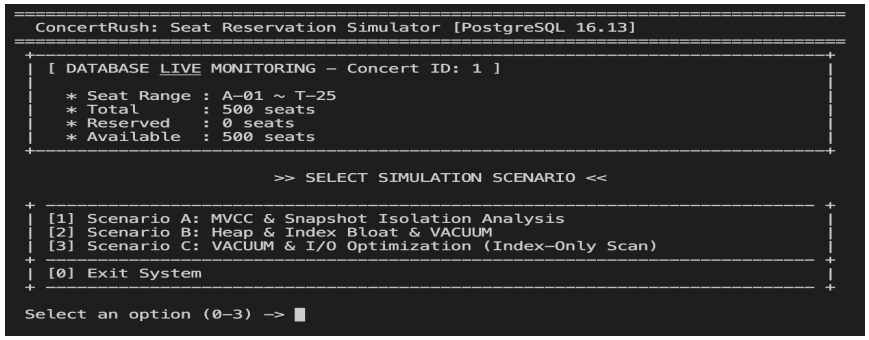
4.1 Overview

`ConcertRush` is a `Python 3` command-line application that simulates a concurrent concert ticket reservation system. It uses `psycopg2` to connect to a local PostgreSQL 16 instance, Python's threading module to simulate concurrent clients. The application is structured around three independent runnable scenarios, each exposing a different layer of PostgreSQL internals.





Seed: 1 concert × 500 seats (A-01 ~ T-25) · PostgreSQL 16 (macOS) · TablePlus for Query Editor



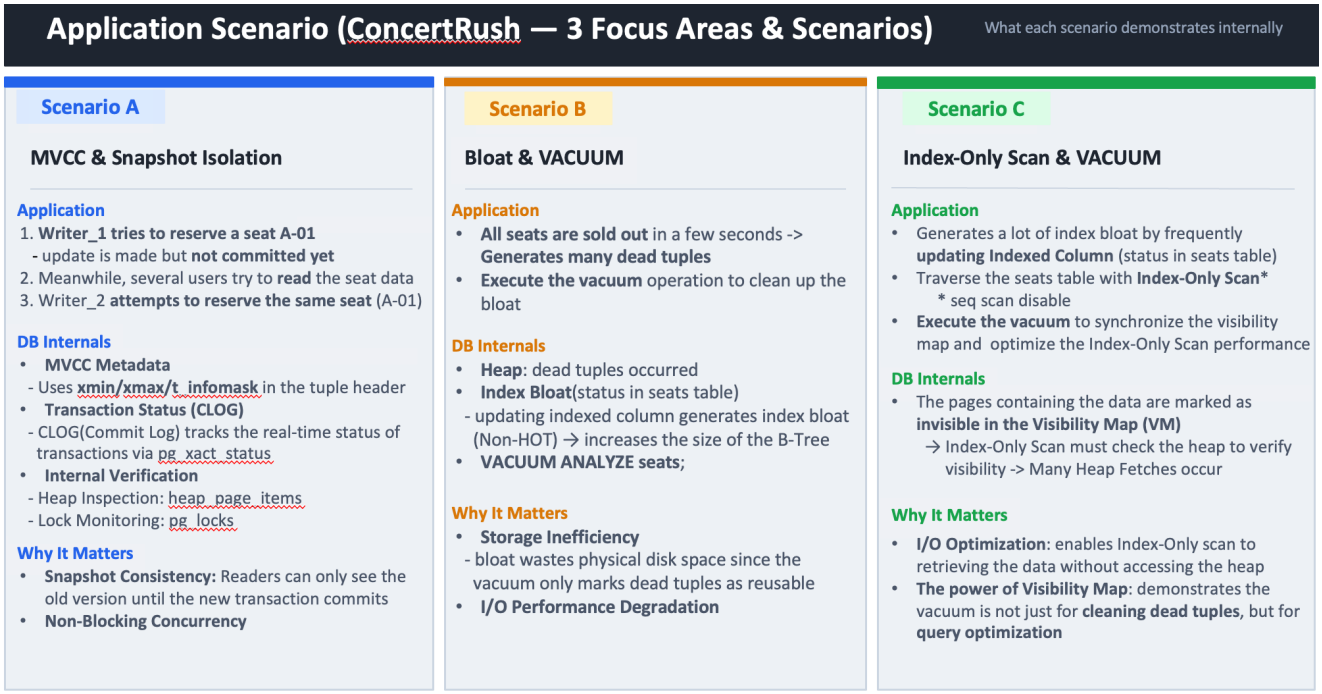
4.2 Detailed Database Schema

```
-- concerts table
CREATE TABLE concerts (
    concert_id   SERIAL PRIMARY KEY,
    name         TEXT NOT NULL,
    venue        TEXT NOT NULL,
    event_date   DATE NOT NULL);

-- seats table (the high-churn target)
CREATE TABLE seats (
    seat_id     SERIAL PRIMARY KEY,
    concert_id  INTEGER REFERENCES concerts(concert_id),
    seat_code   VARCHAR(10) NOT NULL,          -- e.g. 'A-01'
    status      VARCHAR(20) DEFAULT 'available',
    reserved_by TEXT,
    updated_at  TIMESTAMPTZ DEFAULT NOW()
) WITH (fillfactor = 70);                      -- force Non-HOT updates

CREATE INDEX idx_seats_status ON seats(status);
CREATE EXTENSION IF NOT EXISTS pgstattuple;   -- enable pgstattuple extension for bloat
```

5. INTERNALS ↔ APPLICATION BEHAVIOR MAPPINGS

The interaction between the Python application and the PostgreSQL engine was analyzed through three distinct scenarios.

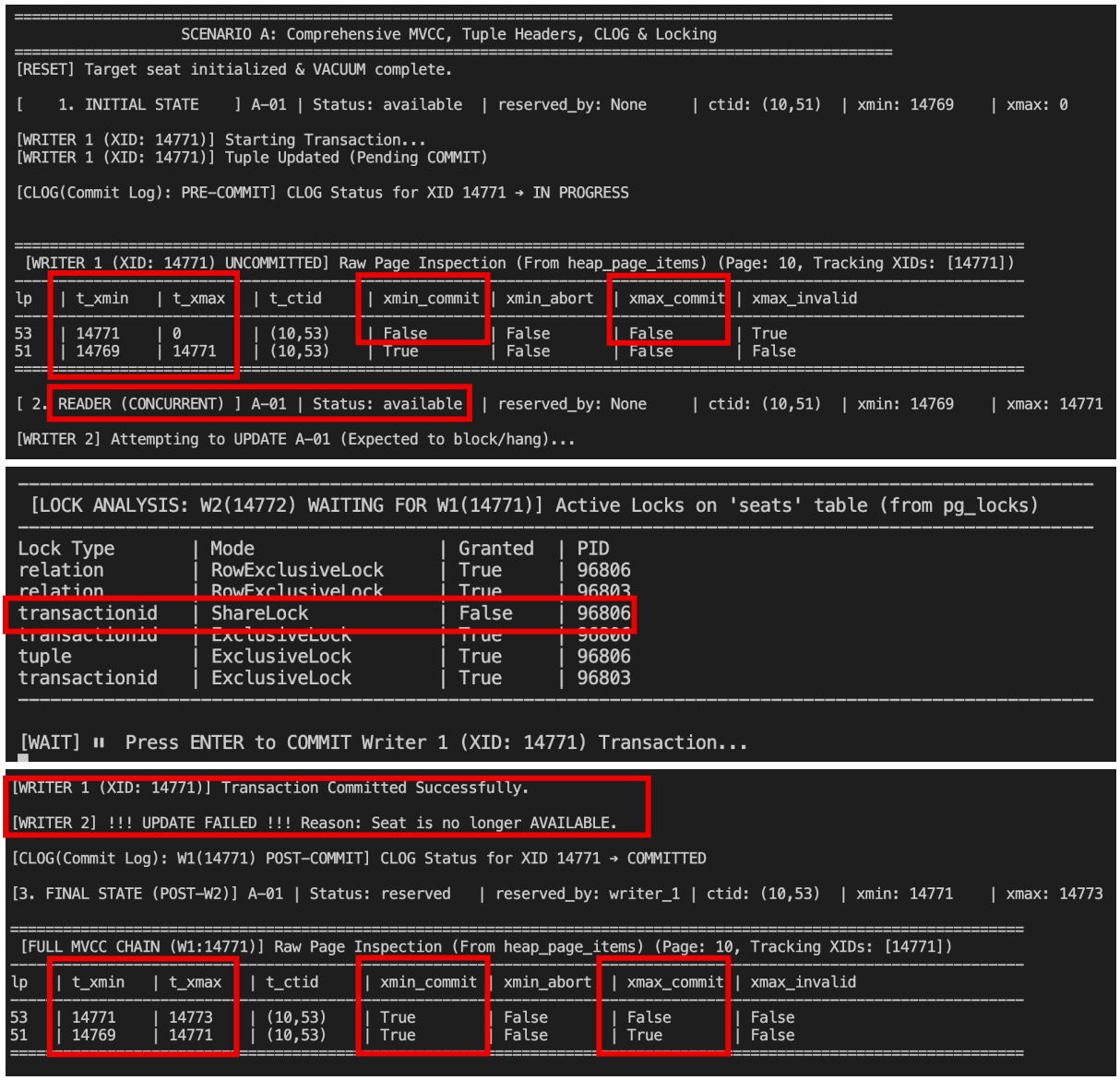
- **Scenario A:** MVCC, Snapshot Isolation, and Lock Analysis
- **Scenario B:** Storage Side-Effects (Heap & Index Bloat) & VACUUM
- **Scenario C:** VACUUM and I/O Optimization (Index-Only Scan)



5.1 Scenario A: MVCC, Snapshot Isolation, and Lock Analysis

Scenario A investigates how PostgreSQL resolves Read-Write and Write-Write conflicts on a single target tuple (Seat A-01).

- **Read-Write Behavior**: When Writer 1 (XID: 14771) updates the seat status from 'available' to 'reserved' but delays the commit, concurrent Readers continue to observe the 'available' status. Inspecting the raw heap page reveals that the old tuple (`xmin`: 14769) remains intact with its `xmax` updated to 14771, while the newly inserted tuple (`xmin`: 14771) has a false `xmin_commit` flag. This empirically proves the snapshot isolation rule: uncommitted writes remain invisible to other active transactions.
- **Write-Write Locking**: A critical addition to this simulation was the introduction of a concurrent Writer 2 attempting to update the exact same seat. As shown in the active lock analysis via `pg_locks`, Writer 1 acquires an `ExclusiveLock` on the specific transaction ID and tuple. Consequently, Writer 2 requests a `ShareLock` on Writer 1's transaction ID but is not granted access (Granted = False), forcing the thread into a waiting state. Once Writer 1 commits successfully, Writer 2's transaction fails because the prerequisite snapshot data has changed, successfully preventing a lost update anomaly.



5.2 Scenario B: Storage Side-Effects (Heap & Index Bloat) & VACUUM

Scenario B models a burst of 500 concurrent seat reservations to monitor storage degradation.

- **Observation:** Because PostgreSQL appends new versions rather than overwriting, 500 updates resulted in the immediate creation of 500 dead tuples. Utilizing the `pgstattuple` diagnostic, the physical storage report showed the `Dead Tuple Ratio` spiking from 0.00% to 31.07%.
- **Index Bloat Analysis:** Due to the intentional prevention of HOT updates, every update generated a new physical location (ctid), forcing the B-Tree index to allocate a new entry for every modified row. This translates directly to increased I/O requirements for subsequent index scans.
- **After VACUUM:** Running VACUUM (`VACUUM ANALYZE`) successfully reclaimed the dead space, and the heap free space has risen. Also, the average `leaf index density` has been reduced from 97.07% to 58.15%

```
================================================================================
                SCENARIO B: Heap & Index Bloat Analysis (MVCC Storage Side-Effects)
================================================================================
[RESET] Disabling Autovacuum (ALTER TABLE seats SET (autovacuum_enabled = false)) and clearing stale data...
[RESET] Running VACUUM to stabilize storage metrics...

================================================================================
 [1. INITIAL BASELINE] PHYSICAL STORAGE DIAGNOSTICS (From pgstattuple & pgstatindex)
--------------------------------------------------------------------------------
  [HEAP]  Dead Tuple Count : 0
  [HEAP]  Dead Tuple Ratio :  0.00%
  [HEAP]  Free Space       : 55,800     bytes
  [INDEX] Total Index Size : 16,384     bytes
  [INDEX] Avg Leaf Density : 41.22%
  [INDEX] Deleted Pages    : 0
================================================================================
```

```
[SUCCESS] Transaction Committed.
[INFO] ALL SEATS ARE SOLD OUT IN SECONDS!
```

```
================================================================================
 [2. POST-WORKLOAD ] PHYSICAL STORAGE DIAGNOSTICS (From pgstattuple & pgstatindex)
--------------------------------------------------------------------------------
  [HEAP]  Dead Tuple Count : 500
  [HEAP]  Dead Tuple Ratio : 31.07%
  [HEAP]  Free Space       : 19,800     bytes
  [INDEX] Total Index Size : 16,384     bytes
  [INDEX] Avg Leaf Density : 96.07%
  [INDEX] Deleted Pages    : 0
================================================================================


[WAIT] ⏸  Press ENTER to perform Manual VACUUM (Reclaim Space)...
```

```
[VACUUM] Reclaiming dead space (with VERBOSE mode)...
[SUCCESS] Manual VACUUM complete.

------------------------------------------------------
 [POSTGRES INTERNAL REPORT]
------------------------------------------------------
 >>> INFO:  vacuuming "concertrush.public.seats"
 >>> INFO:  finished vacuuming "concertrush.public.seats": index scans: 1
pages: 0 removed, 11 remain, 11 scanned (100.00% of total)
tuples: 500 removed, 500 remain, 0 are dead but not yet removable
removable cutoff: 14778, which was 0 XIDs old when operation ended
new relfrozenxid: 14778, which is 3 XIDs ahead of previous value
frozen: 11 pages from table (100.00% of total) had 500 tuples frozen
index scan needed: 11 pages from table (100.00% of total) had 500 dead item identifiers removed
```

```
================================================================================
 [3. POST-VACUUM (Space Reclaimed)] PHYSICAL STORAGE DIAGNOSTICS (From pgstattuple & pgstatindex)
--------------------------------------------------------------------------------
  [HEAP]  Dead Tuple Count : 0
  [HEAP]  Dead Tuple Ratio :  0.00%
  [HEAP]  Free Space       : 49,808     bytes
  [INDEX] Total Index Size : 16,384     bytes
  [INDEX] Avg Leaf Density : 58.15%
  [INDEX] Deleted Pages    : 0
================================================================================


[CLEANUP] Autovacuum re-enabled.
```

5.3 Scenario C: VACUUM and I/O Optimization (Index-Only Scan)

Scenario C measures the query performance impact of PostgreSQL's garbage collection process, VACUUM.

- **Pre-VACUUM State:** Following the mass update, the `Visibility Map (VM)` marked all pages (Page 0 through 10) as Dirty (`FALSE`). When executing an `Index-Only Scan` to find 'available' seats, the query planner reported 500 Heap Fetches with an execution time of 0.624 ms. Because the `VM` bits were dirty, the database engine was forced to fetch the actual heap pages to verify tuple visibility, nullifying the I/O benefits of the index.
- **Post-VACUUM State:** Running a manual `VACUUM ANALYZE` successfully reclaimed the dead space (Free Space increased to 49,808 bytes) and updated the `Visibility Map`, marking all pages as Clean (`TRUE`). Executing the exact same query post-VACUUM yielded 0 Heap Fetches and dropped the execution time to 0.111 ms. This demonstrates an 82% reduction in execution time, proving that proper vacuuming is strictly required to unlock the performance benefits of Index-Only Scans.

```
==========================================================================
                SCENARIO C: Index-Only Scan Optimization via Visibility Map (VM)
==========================================================================

[ACTION] VM bits are being updated by modifying indexed column 'status'...
--------------------------------------------------------------------------
 [UPDATE OPERATION] Target: seats table | Filter: concert_id=1
 Result: 500 index column rows updated.
 Impact: All tuples in Page 0 marked as 'Dirty' in Visibility Map.
--------------------------------------------------------------------------

 [ PRE-VACUUM STATE ] VISIBILITY MAP (VM) SNAPSHOT (From pg_visibility)
-------------------------------------------------------
Page (Block) No.          | All Visible
-------------------------------------------------------
Page 0                    | FALSE (Dirty)
Page 1                    | FALSE (Dirty)
Page 2                    | FALSE (Dirty)
Page 3                    | FALSE (Dirty)
Page 4                    | FALSE (Dirty)
Page 5                    | FALSE (Dirty)
Page 6                    | FALSE (Dirty)
Page 7                    | FALSE (Dirty)
Page 8                    | FALSE (Dirty)
Page 9                    | FALSE (Dirty)
Page 10                   | FALSE (Dirty)
-------------------------------------------------------
```

< VM Status: Before VACUUM>

```
[VACUUM] Running VACUUM ANALYZE to update VM bits...
[SUCCESS] Visibility Map has been synchronized with physical storage.

 [ POST-VACUUM STATE ] VISIBILITY MAP (VM) SNAPSHOT (From pg_visibility)
-------------------------------------------------------
Page (Block) No.          | All Visible
-------------------------------------------------------
Page 0                    | TRUE (Clean)
Page 1                    | TRUE (Clean)
Page 2                    | TRUE (Clean)
Page 3                    | TRUE (Clean)
Page 4                    | TRUE (Clean)
Page 5                    | TRUE (Clean)
Page 6                    | TRUE (Clean)
Page 7                    | TRUE (Clean)
Page 8                    | TRUE (Clean)
Page 9                    | TRUE (Clean)
Page 10                   | TRUE (Clean)
-------------------------------------------------------
```

<VM Status: After VACUUM>

```
[QUERY PREPARATION]
 - Target Query: SELECT status
    FROM seats
    WHERE concert_id = %s AND status = 'available';
 - Optimization: Setting enable_seqscan = OFF (Forcing Index Usage)
=============================================================================================================================
 [DIRTY VM: PERFORMANCE TEST] PERFORMANCE ANALYSIS
-----------------------------------------------------------------------------------------------------------------------------
 Scan Methodology : Index Only Scan
 Heap Fetches     : 500
 Execution Time   : 0.624 ms
-----------------------------------------------------------------------------------------------------------------------------
  RAW EXPLAIN PLAN DETAILS:
    Index Only Scan using idx_seats_concert_status on seats  (cost=0.15..4.17 rows=1 width=9) (actual time=0.140..0.293 rows=500 loops=1)
    Index Cond: ((concert_id = 1) AND (status = 'available'::text))
    Heap Fetches: 500
    Buffers: shared hit=13
    Planning:
    Buffers: shared hit=108
    Planning Time: 0.972 ms
    Execution Time: 0.624 ms
=============================================================================================================================
```

<I/O Performance: Before VACUUM>

```
[QUERY PREPARATION]
 - Target Query: SELECT status
    FROM seats
    WHERE concert_id = %s AND status = 'available';
 - Optimization: Setting enable_seqscan = OFF (Forcing Index Usage)
=============================================================================================================================
 [CLEAN VM: PERFORMANCE TEST] PERFORMANCE ANALYSIS
-----------------------------------------------------------------------------------------------------------------------------
 Scan Methodology : Index Only Scan
 Heap Fetches     : 0
 Execution Time   : 0.111 ms
-----------------------------------------------------------------------------------------------------------------------------
  RAW EXPLAIN PLAN DETAILS:
    Index Only Scan using idx_seats_concert_status on seats  (cost=0.15..18.15 rows=500 width=10) (actual time=0.045..0.074 rows=500 loops=1)
    Index Cond: ((concert_id = 1) AND (status = 'available'::text))
    Heap Fetches: 0
    Buffers: shared hit=2
    Planning:
    Buffers: shared hit=108
    Planning Time: 0.451 ms
    Execution Time: 0.111 ms
=============================================================================================================================
```

<I/O Performance: After VACUUM>

6. COMPARISON WITH MYSQL AND MONGODB

- **PostgreSQL vs. MySQL (InnoDB)**: PostgreSQL's append-only MVCC stores older tuple versions in the main heap, causing the table size to grow continuously until vacuumed. InnoDB utilizes an update-in-place strategy, moving old data to a separate Undo Log. While InnoDB is less susceptible to table bloat in high-update workloads, its reliance on undo logs can cause severe performance bottlenecks for long-running read transactions, a problem PostgreSQL avoids entirely.

- **PostgreSQL vs. MongoDB**: MongoDB, a NoSQL database using the WiredTiger engine, also utilizes MVCC but applies locking at the document level rather than the relational row level. While MongoDB excels at horizontal scalability and handling unstructured JSON data, PostgreSQL provides much stricter enforcement of relational integrity and finer-grained locking controls (e.g., tuple-level Exclusive Locks observed in Scenario A). For a seat reservation system — where the data is highly relational, seat-level ACID transactions are a hard requirement, and complex join queries are common — PostgreSQL is the more appropriate choice. MongoDB's WiredTiger storage engine does not suffer from heap bloat in the same way as PostgreSQL, but its MVCC internals are less transparent to application developers.

Feature	PostgreSQL	MySQL InnoDB	MongoDB
Data Model	Relational	Relational	Document (NoSQL)
MVCC Strategy	Heap-based versioning	Undo-log versioning	WiredTiger MVCC
Heap Bloat Risk	Yes (VACUUM needed)	No	No
Index-Only Scan	Yes (via VM)	Limited (covering index)	No direct equiv.
Multi-row Transactions	Native, full ACID	Native, full ACID	Added v4.0, higher cost
Horizontal Scaling	Via Citus extension	Via ProxySQL/Vitess	Native sharding
Schema Flexibility	Rigid (DDL)	Rigid (DDL)	Flexible (schemaless)

7. LIMITATIONS & LESSONS LEARNED

- **Limitations & Future Improvements:** The current scope of *ConcertRush* evaluates concurrency and lock contention within a single-node PostgreSQL instance. It does not account for the complexities of distributed deadlocks or replication lag that would be present in a multi-node, highly available production cluster.
    - **Future extensions to `ConcertRush` could include**: (1) a web-based front end (FastAPI + React) to simulate realistic multi-client concurrency over network connections; (2) comparative Scenario B measurements against a MySQL InnoDB target database to directly contrast heap bloat absence with undo log growth; and (3) integration with TimescaleDB to model time-series reservation analytics over the same seat inventory.
- **Challenges:** During `Scenario B`, PostgreSQL's aggressive background `autovacuum` daemon frequently reclaimed dead tuples before the Python script could measure the bloat. This was resolved by dynamically executing `ALTER TABLE seats SET (autovacuum_enabled = false)` for the duration of the test block. PostgreSQL performs HOT updates by default when the updated column is not indexed, and the new tuple fits on the same page. This suppressed the Non-HOT bloat that Scenario B was designed to demonstrate. The solution was to (a) add an explicit index on the status column (defeating HOT eligibility) and (b) set fillfactor=70 to ensure pages fill quickly, forcing non-HOT updates reliably regardless of tuple size.
- **Lessons Learned:** PostgreSQL's append-only MVCC design is simultaneously its greatest strength (lock-free reads) and its greatest operational challenge (bloat, VACUUM dependency). No design decision is free. The empirical evidence proved that without aggressive and properly tuned vacuuming strategies, the database will incur hidden I/O penalties (Heap Fetches), rendering secondary indexes highly inefficient.

8. PROJECT LINKS

- **GitHub repository link**: https://github.com/SongQoo/DSCI551_Project

REFERENCES

[1] PostgreSQL Global Development Group. PostgreSQL 16 Documentation, Ch. 13: Concurrency Control. https://www.postgresql.org/docs/current/mvcc.html
[2] Suzuki, H. (2024). The Internals of PostgreSQL, Ch. 5–6. https://www.interdb.jp/pg/
[3] PostgreSQL Global Development Group. pgstattuple. https://www.postgresql.org/docs/current/pgstattuple.html
[4] Momjian, B. (2021). PostgreSQL MVCC Unmasked. https://momjian.us/main/presentations/internals.html
[5] CYBERTEC. (2023). HOT Updates in PostgreSQL. https://www.cybertec-postgresql.com/en/hot-updates-in-postgresql/